

5. Канално ниво.

**Кадри, предаване, грешки,
номерация, прозорци**

Какво ще научим

- основните функции на каналния слой;
- надеждно и ненадеждно предаване;
- нормиране на кадъра (frame);
- откриване на грешки в кадрите;
- протоколи HDLC и PPP(оЕ);
- MPLS – протокол на 2.5 слой.

Основни функции

Каналното ниво има **три основни функции**:

- да осигури подходящ интерфейс на по-горното мрежово ниво,
- да открива грешки по време на предаването и
- да управлява информационния обмен между два съседни възела в мрежата.

Данните за каналното ниво представляват последователност от **кадри** (frame).

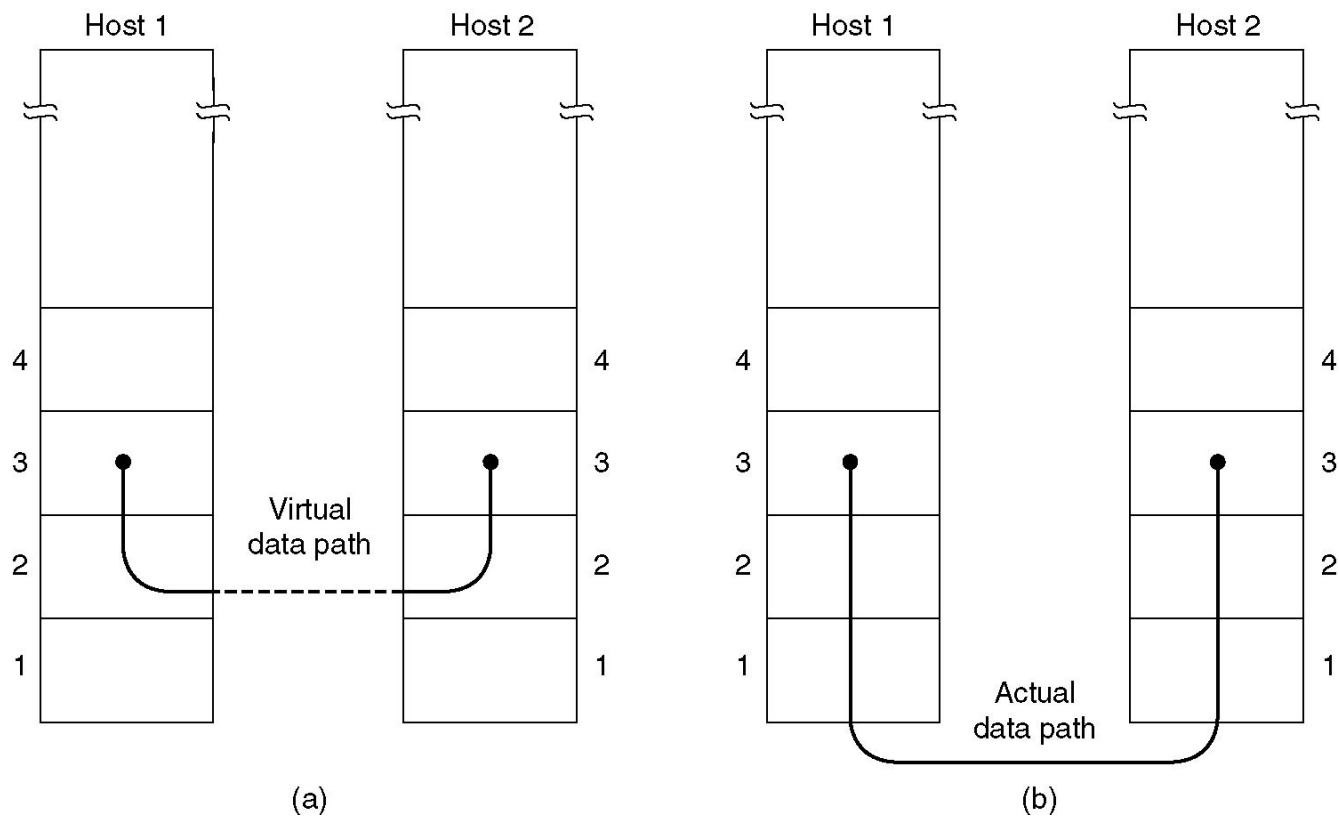
Каналите са три вида - **симплексни, полудуплексни и дуплексни**.

Дуплексните канали позволяват едновременно предаване в двете посоки.

Полудуплексните канали позволяват предаване и в двете посоки, но в даден момент може да се предава само в една посока.

Симплексните канали позволяват предаване само в една посока.

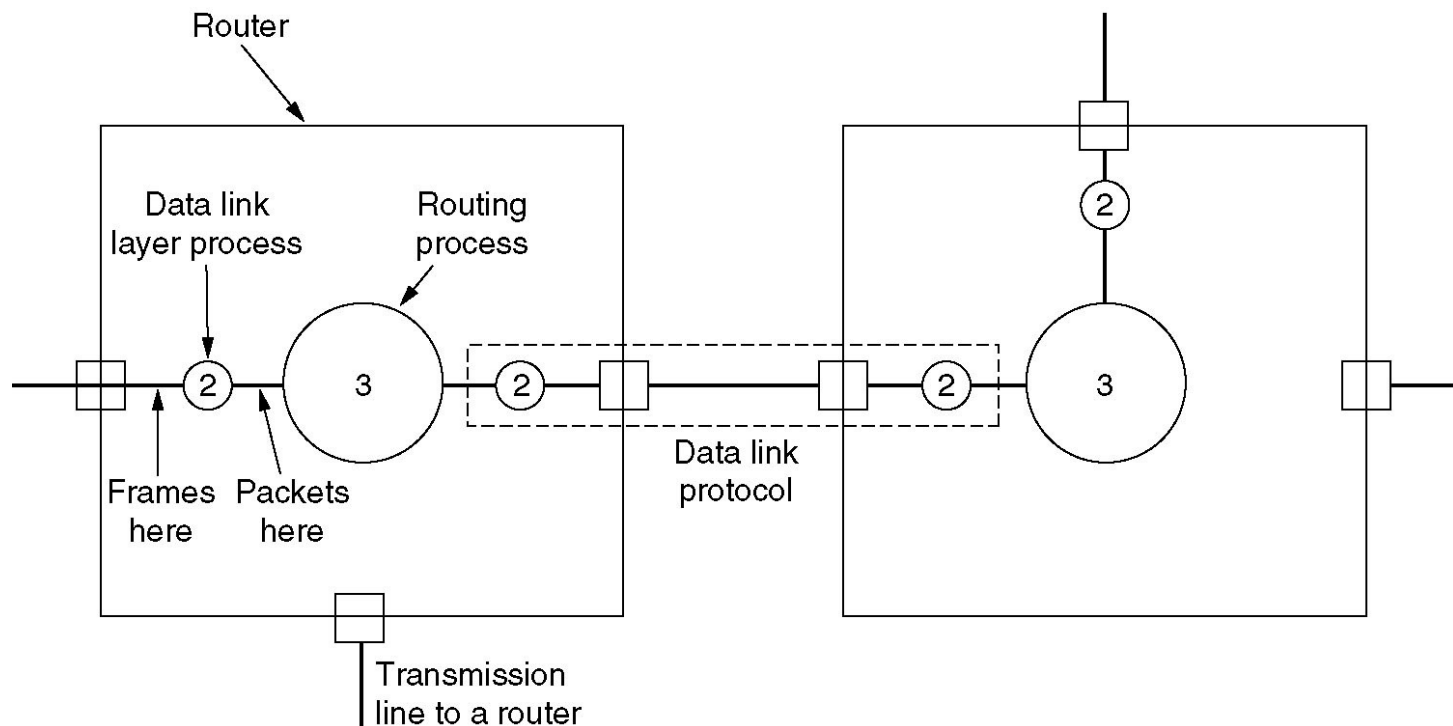
Основни функции



(a) Логическа комуникация.

(b) Действителна комуникация.

Основни функции



Ролята на каналния слой между две съседни машини - възли.

Основни услуги

Най-общата услуга - **прехвърляне на данни (надеждно или best effort, но изчистено от грешки)** между мрежовото ниво на източника и мрежовото ниво на получателя (всъщност самото предаване се извършва от физическото ниво, но това остава невидимо за мрежовото ниво).

Основните варианти на тази услуга са:

- непотвърдено, без установяване на сесия (**Unacknowledged connectionless service**),
- потвърдено, без установяване на сесия (**Acknowledged connectionless service**) и
- потвърдено, с установяване на сесия (**Acknowledged connection-oriented service**).

Основни услуги

Непотвърденото, без установяване на сесия

Източникът изпраща независими кадри към получателя, без получателя да ги потвърждава. Няма установяване на сесия между двете машини.

Ако един кадър се загуби поради шум в линията, каналното ниво не прави опит да възстанови този кадър. Това обслужване е подходящо при канали с много малка честота на грешките, което позволява функциите по възстановяване на загубената информация да се поемат от по-горни нива в йерархията.

Основни услуги

Такова обслужване се реализира в повечето LAN.

То също се използва когато навременното получаване на кадрите е по-важно от тяхната достоверност видео, глас в реално време.

При потвърденото, без установяване на сесия отново не се установява сесия между източника и получателя, но получаването на всеки кадър се потвърждава самостоятелно от получателя. Това дава възможност за повторно изпращане на непотвърдените кадри.

Основни услуги

Потвърждаването на получената информация е функция на транспортното ниво, но там то се отнася до последователности от сегменти.

Потвърждаването на каналното ниво има смисъл при ненадеждна комуникационна среда, каквато е **безжичната**, тъй като повторно ще се предават само непотвърдените кадри.

Основни услуги. Connection Oriented.

Потвърденото, с установяване на сесия има три фази.

Първата фаза се установява сесия и се заделят необходимите ресурси (локални буфери, броячи и т.н.).

Втората фаза се изпращат кадрите.

Третата фаза се освобождават ангажираните ресурси.

Гарантира се не само успешното предаване на кадъра, но и последователността в която се предават кадрите.

Управление на потока (Flow Control)

Друг проблем, който е свързан с управлението на обмена на канално ниво е **източникът да изпраща кадри по-бързо, отколкото те могат да бъдат приети** от получателя.

За целта се въвеждат **механизми за управление на потока** от кадри, който осигурява обратна информация на източника за темпа на предаване.

Обикновено механизмите по управление на обмена се изпълняват в транспортния слой над цели масиви от данни, обхващащи последователност от кадри.

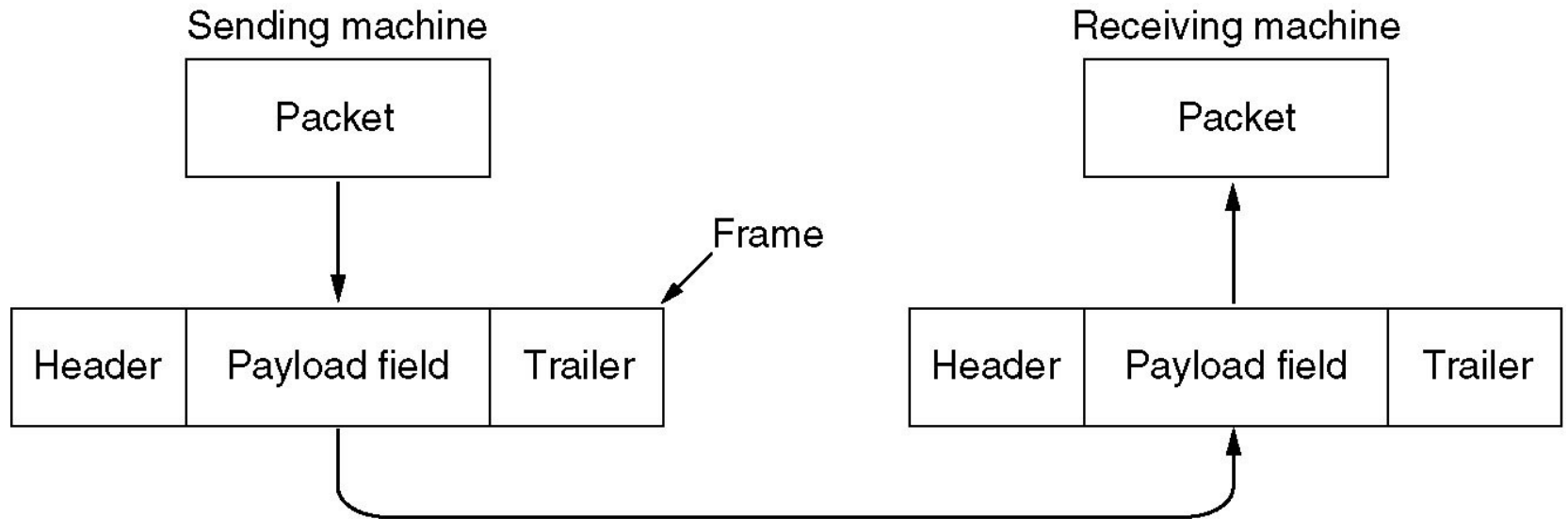
Формиране на кадри

Каналното ниво взима пакетите, които му се подават от мрежовото ниво и ги опакова в кадри.

Всеки кадър се състои от заглавна част (**header**), поле за данни (**data** или **payload**), което съдържа мрежовия пакет и опашка (**trailer**). Дължината на кадъра обикновено е ограничена отгоре.

Физическото ниво възприема информацията от каналното ниво като поток от битове, без да се интересува от нейната структура.

Формиране на кадри



Прехвърляне на данни между мрежовите нива на източник и получател (две съседни машини - възли). Пакети и кадри.

Формиране на кадри

Получателят идентифицира в потока от битове кадрите и въз основа на служебната информация в тях ги **контролира за грешки**.

За целта опашката на кадъра съдържа **контролна сума** (обикновено 2 байта), която се изчислява върху останалата част от кадъра преди той да бъде предаден.

Когато кадърът пристигне при получателя, контролната сума се преизчислява и ако тя е различна от предадената контролна сума, то получателят отхвърля кадъра и евентуално изпраща съобщение за грешка към източника.

Формиране на кадри

Разделянето на потока от битове на кадри не е тривиална задача.

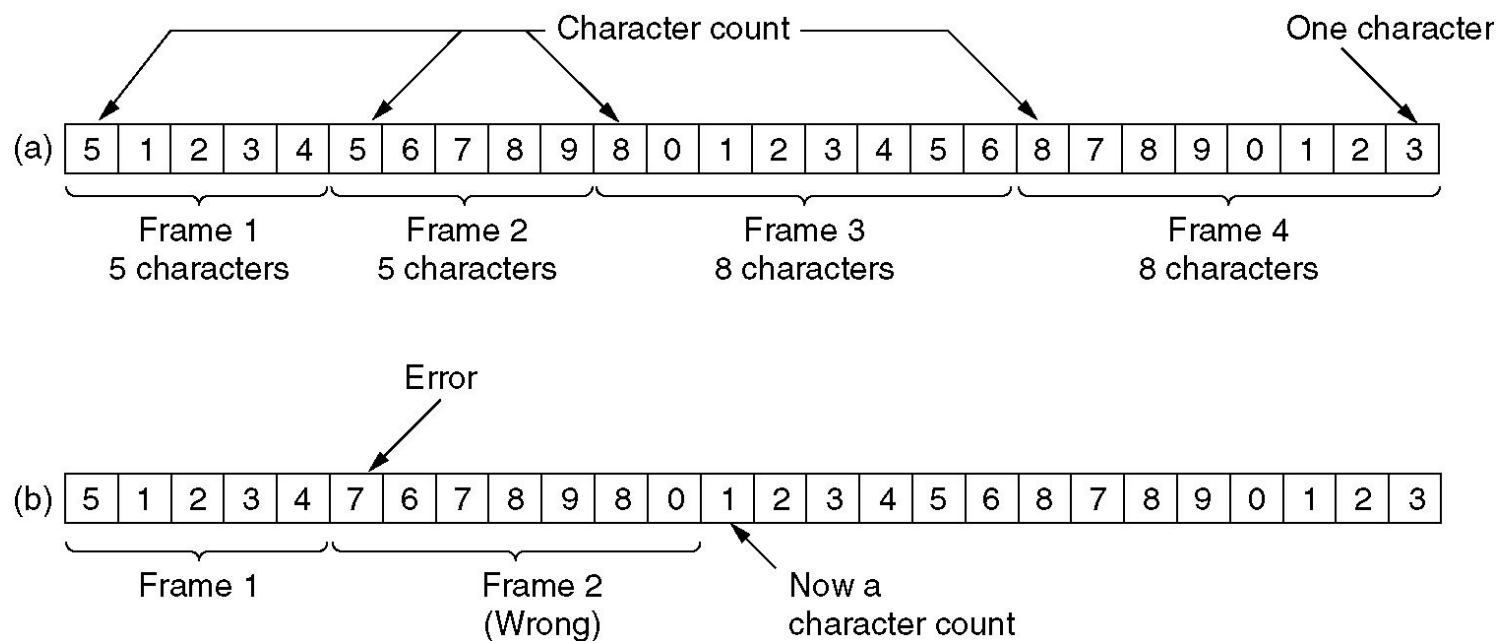
Един начин е **между всеки два кадъра да се въведе времеви интервал**. Този подход е твърде несигурен, тъй като времевите интервали могат да се променят по време на предаването.

Понастоящем основно се използват **три метода**.

При първия метод е **броене на отделните символи**. В заглавието на кадъра се указва броя на символите в целия кадър.

Основният проблем на този метод е, че **броят на символите може да бъде сгрешен по време на предаването**, при което получателят ще загуби синхронизация и няма да може да определи началото на следващия кадър. Затова **не се използва**.

Броене на символи



Поток от четири кадъра: 5, 5, 8, 8 символа. (a) Без грешки. (b) Грешка: число 5 във втори става 7. Губи се синхронизация.

Формиране на кадри

Втори метод в началото и края на кадъра се вмъкват специални служебни символи - **STX** (start of text) за начало на кадър и **ETX** (end of text) за край на кадър, които маркират границите на кадъра. Техниката е известна като **вмъкване на символи** (**byte stuffing, character stuffing**).

Възможно е служебните символи да се срещат като битови последователности в оригиналните данни. За решение на този проблем се въвежда друг служебен символ **ESC** (**escape**), който се вмъква преди всяко срещане на служебен символ (STX, ETX, ESC) в данните. Например, ако потокът, предаван от мрежовия слой на източника е **A STX ESC B**, той ще се **преобразува** в **A ESC STX ESC ESC B**.

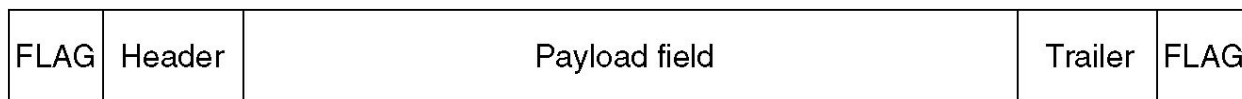
Формиране на кадри

Каналното ниво на получателя ще премахне символите ESC (като при два последователни ESC, единият се запазва), преди да предаде данните на мрежовото ниво на получателя.

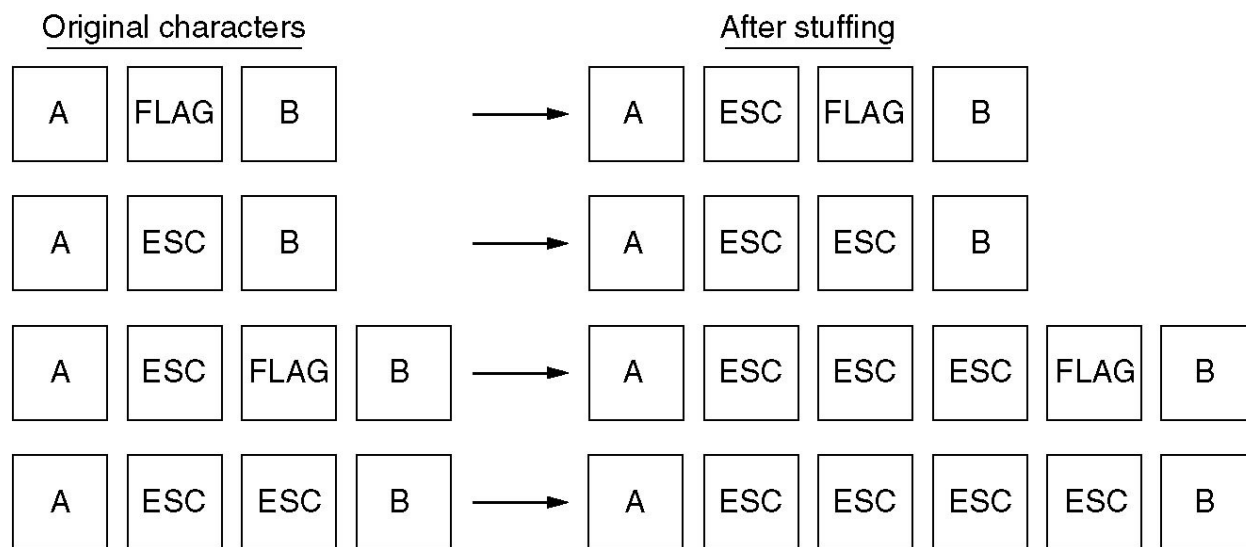
При по-новите протоколи се използва един и същ символ за маркиране на началото и края на кадъра - **флаг**.

Недостатъкът на този метод е, че той се обвързва с 8-битови символи, кодирани в ASCII.

Вмъкване на символи



(a)



(b)

(a) кадър, ограничен от флагови байтове.

(b) Последователност от байтове преди и след вмъкване.

Формиране на кадри

С развитието на мрежите стана възможно кадрите да съдържат произволно цяло число битове. За такива кадри се използва **третия метод**, при който началото и края на всеки кадър се маркира с битовата последователност **01111110**, наречена **флагов байт**.

За да се предотврати погрешното определяне на граница на кадър, ако тази последователност от битове се срещне в данните на кадъра, след всеки 5 единици в данните източникът добавя по една нула. Техниката се нарича **вмъкване на битове (bit stuffing)**.

Каналното ниво на получателя премахва нулата след всеки 5 единици в данните, преди да ги подаде на мрежовото ниво.

За постигане на допълнителна сигурност при много протоколи броенето на символи се комбинира с някой от другите два метода.

Флагов байт

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0



Stuffed bits

(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Bit stuffing

(a) Оригинални данни

(b) Как данните изглеждат по линията

(c) Данните, съхранени в паметта на приемника след destuffing

Процедури за надеждна работа на канала

Функциите на каналния слой се реализират в **адаптер, предимно хардуерно**: специализирани интегрални схеми за управление (**ASIC**) и програмен код, “прогорен” в EEPROM или записан във Flash памет (firmware).

В адаптера е реализиран **буфер**, в който се записват кадрите, докато изчакват да бъдат предадени нататък.

Кадърът преседява в буфера, докато не се увери, че отсрещната страна го е получила.

Да приемем, че **източник А** изпраща кадър към **В**, но той изобщо не стига до там. Пет причини за това:

- 1) Адаптер **А** дефектен, не излъчва правилен сигнал;
- 2) “Счупен” канал – жица и т.н.;
- 3) **В** не съществува;
- 4) **В** няма свободен буфер;
- 5) Кадърът постъпва в буфера на **В**.

Процедури за надеждна работа на канала

(Ако се иска потвърждение.)

A може да получи отговор единствено при 5). При изпращане на кадъра **A** включва брояч на време - таймер. Чака отговор до определено време – **timeout**.

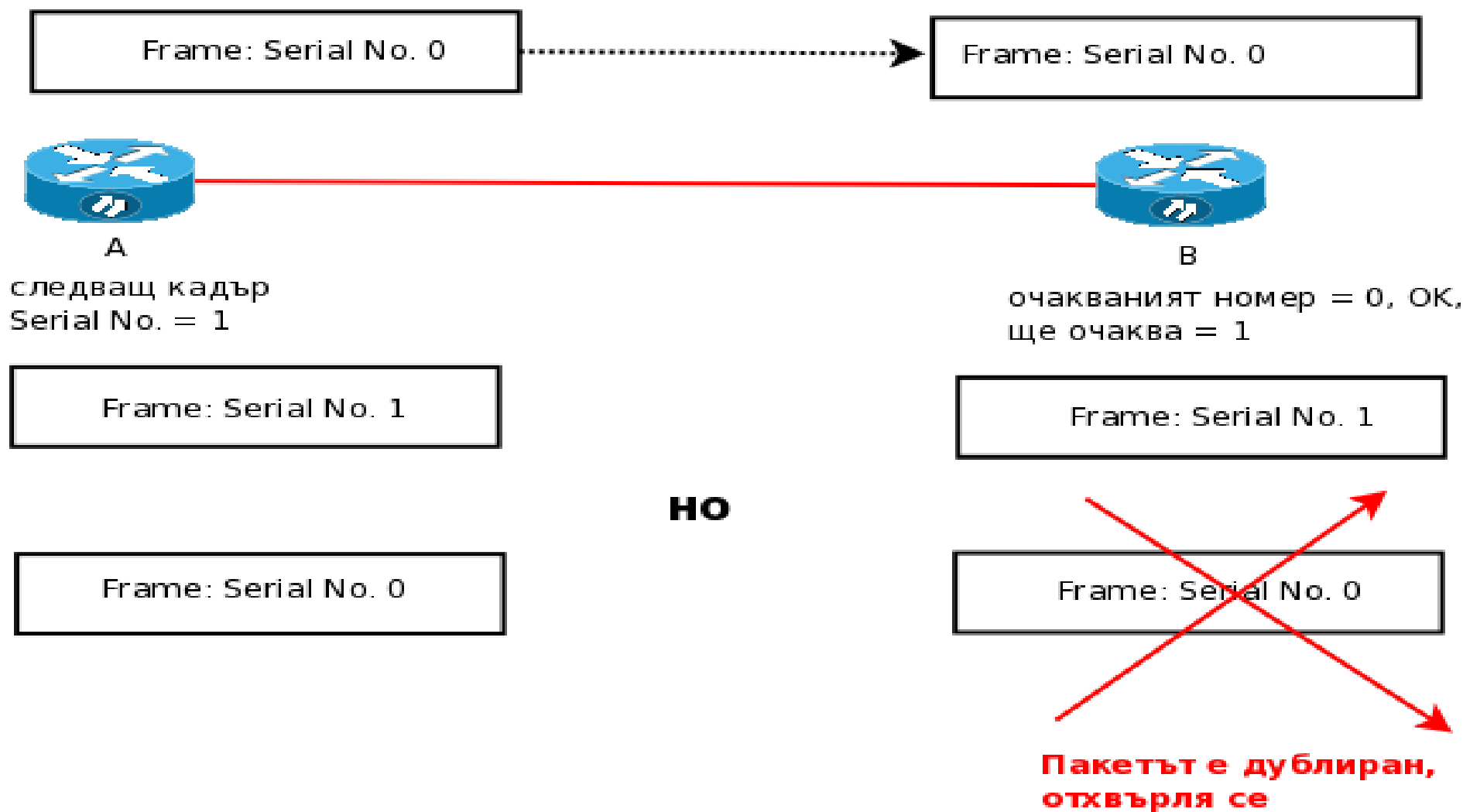
timeout трябва да е по-голямо от времето за предаване на кадъра, обработката му в приемника и получаване на потвърждение.

Ако кадърът не се потвърди в рамките на това време, то **A** предава кадърът отново.

Възможно е **A** да изпрати кадър към **B**, този кадър да се получи в **B**, но потвърждението да се изгуби.

Процедури за надеждна работа на канала. Пореден номер.

Serial No. = 0 or 1 (1-bit)



Откриване на грешки в кадрите

Горните методи си имат недостатъци. Затова...

CRC (*cyclic redundancy check* — проверка на цикличния остатък)

Алгоритъм за проверка за грешки при предаване и съхранение на данни чрез използване на **контролна сума** (контролно число, CRC сума).

Устройството-източник изчислява CRC-сумата на данните, които следва да бъдат проверявани и я изпраща или записва със самите данни.

Устройството-получател извършва същото изчисление след прочитане на данните и контролната сума, и установява тяхната автентичност чрез сравнение на записаната CRC сума и новоизчислената CRC сума.

Видове CRC-та

CRC-16-CCITT = $x^{16} + x^{12} + x^5 + 1$ (Bluetooth, XMODEM, HDLC, PPP)

CRC-32-IEEE 802.3 = $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

CRC-64-ISO = $x^{64} + x^4 + x^3 + x + 1$ (HDLC — ISO 3309)

Тези кодове са разработени отдавна, оптимизирани и вкарани в хардуерни схеми.

CRC се изчислява в движение и се слага в края на кадъра като **FCS** (Frame Control Sum)

Modulo 2 Аритметика

Изпълнява се цифра по цифра върху двоични числа. Няма “преноси” и “заеми”.

Събиране

Използва се **exclusive OR (XOR)** функцията:

A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

Пример:

$$\begin{array}{r} (X) \ 10110100 \\ (Y) \ 00101010 \ + \\ \hline (Z) \ 10011110 \end{array}$$

Modulo 2 Аритметика (Изваждане)

Дава същите резултати като събиране:

(X) 10110100

(Z) 10011110 -

(Y) 00101010

$$X + Y = Z, \Rightarrow Y = Z - X.$$

От примера следва и: $Y = Z + X$.

Modulo 2 Аритметика (Деление)

Подобно на аритметическото деление на двоични числа. Пак използваме Modulo 2 изваждане.

$$\begin{array}{r} 10001 \text{ *остатък* } 100 \\ \text{-----} \\ 10011 | 100100111 \\ 10011 \\ \hline 10111 \\ 10011 \\ \hline 100 \end{array}$$

Имаме $X/Y = Y/X$. Напр.:

$$\begin{array}{r} 1 \text{ *остатък* } 1011 \\ \text{-----} \\ 11001 | 10010 \\ 11001 \\ \hline 1011 \end{array}$$

$$\begin{array}{r} 1 \text{ *остатък* } 1011 \\ \text{-----} \\ 10010 | 11001 \\ 10010 \\ \hline 1011 \end{array}$$

Пример на CRC изчисление

Подател и получател се уговарят за генератор на контролната сума, **полином $G(x)$** . Най-старшия и най-младшия бит трябва да са 1.

За да изчислим контролната сума на **кадър с t бита**, полинома **$M(x)$** , кадърът трябва да е по-дълъг от полинома на генератора.

Идеята е към края на кадъра се да се прибави контролна сума, така че полиномът, който представя **“checksummed” кадър**, да е делим на $G(x)$.

Когато получателят приеме **“checksummed” кадър**, той се опитва да го раздели на $G(x)$. **Ако се получи остатък, значи има грешка.**

Пример на CRC изчисление

Алгоритъмът за изчисляване на контролната сума е следния:

1. Нека r е степента на $G(x)$. Прикрепяме r нулеви бита към “младшия” край на кадъра. Така той вече съдържа $m + r$

Бита и съответства на полинома $x^r M(x)$.

2. Делим низа от битовете, съответстващ на $G(x)$, на битовия низ, съответстващ на $x^r M(x)$ с помощта на “сума по модул 2” (modulo 2) делене.

3. Изваждаме **остатъка** (който винаги е $\leq r$ бита) от битовия низ, съответстващ на $x^r M(x)$ с помощта на **modulo 2 изваждане**. Резултатът е **checksummed frame**, който ще се предаде, т.е полинома $T(x)$.

В следващия пример имаме кадър 1101011011 и генератор:

$$G(x) = x^4 + x + 1$$

Пример на CRC изчисление

Алгоритъмът за изчисляване на контролната сума е следния:

1. Нека r е степента на $G(x)$. Прикрепяме r нулеви бита към “младшия” край на кадъра. Така той вече съдържа $m + r$

Бита и съответства на полинома $x^r M(x)$.

2. Делим низа от битовете, съответстващ на $G(x)$, на битовия низ, съответстващ на $x^r M(x)$ с помощта на “сума по модул 2” (modulo 2) делене.

3. Изваждаме **остатъка** (който винаги е $\leq r$ бита) от битовия низ, съответстващ на $x^r M(x)$ с помощта на **modulo 2 изваждане**. Резултатът е **checksummed frame**, който ще се предаде, т.е полинома $T(x)$.

В следващия пример имаме кадър 1101011011 и генератор:

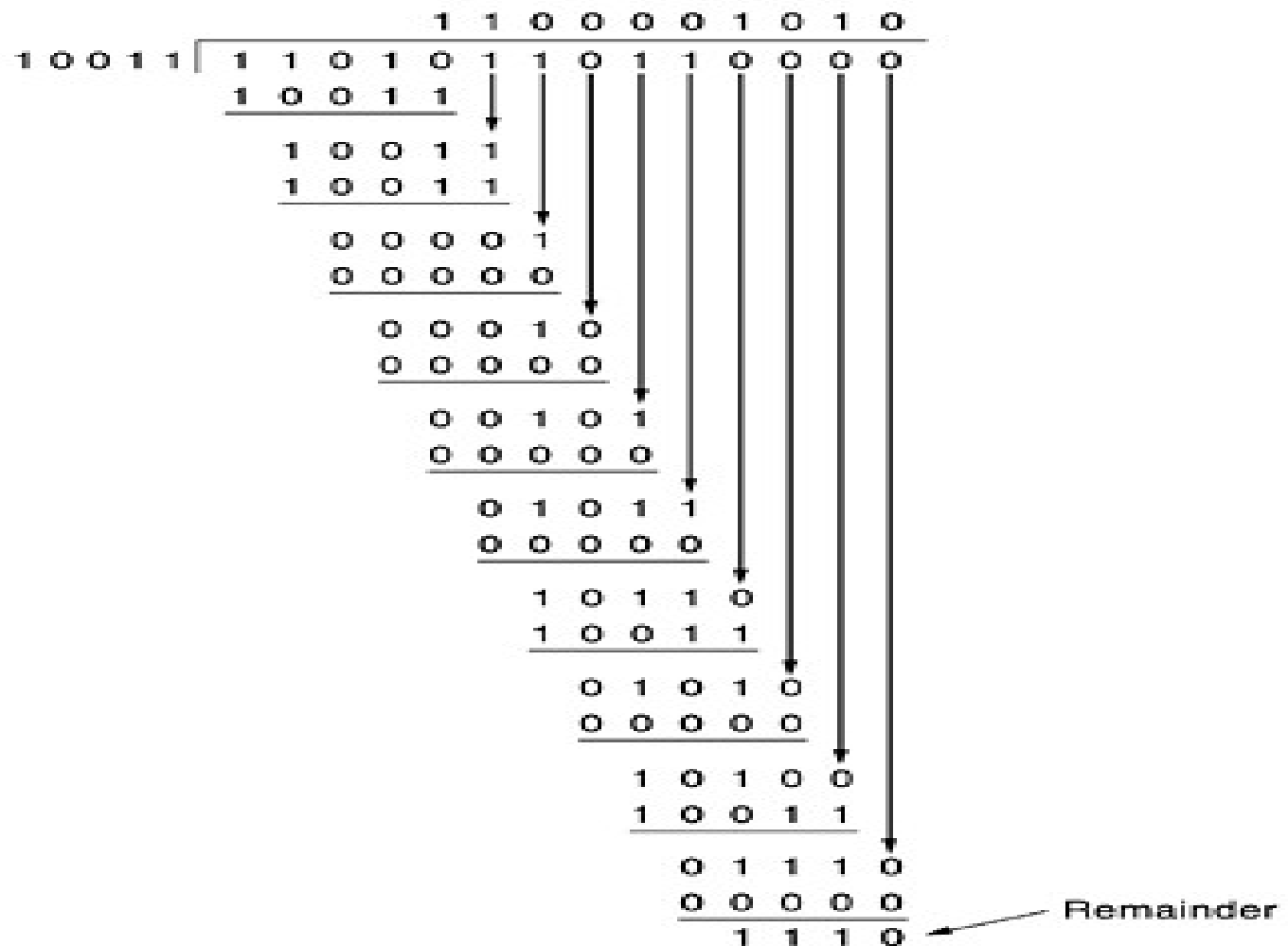
$$G(x) = x^4 + x + 1$$

Пример на CRC изчисление

Frame : 1 1 0 1 0 1 1 0 1 1

Generator: 1 0 0 1 1

Message after 4 zero bits are appended: 1 1 0 1 0 1 1 0 1 1 0 0 0 0



Протоколи с прозорци (Sliding Window Protocols)

Плъзгащият се прозорец (Sliding window) се използва за по-ефективно предаване от протоколите със сесии (connection oriented):

- на 2 слой - Point-to-Point protocol (PPP);
- на 4 слой TCP.

Прозорецът се прилага при двупосочно предаване (full duplex). На 2 слой – два типа кадри:

1. Data
2. ACK (потвърждение - поредният номер на последния получен без грешка кадър)

Sliding Window Protocols

Предавател и приемник поддържат `“прозорец” на потвържденията:

- предавател – стойността на очакваното потвърждение когато получи потвърждение от приемника, прозорецът „напредва“;
- приемник – стойността на номера на очаквания кадър. Когато получи очакваният кадър, приемникът „премества наред“ прозореца.

Stop-And-Wait (1-bit Window)

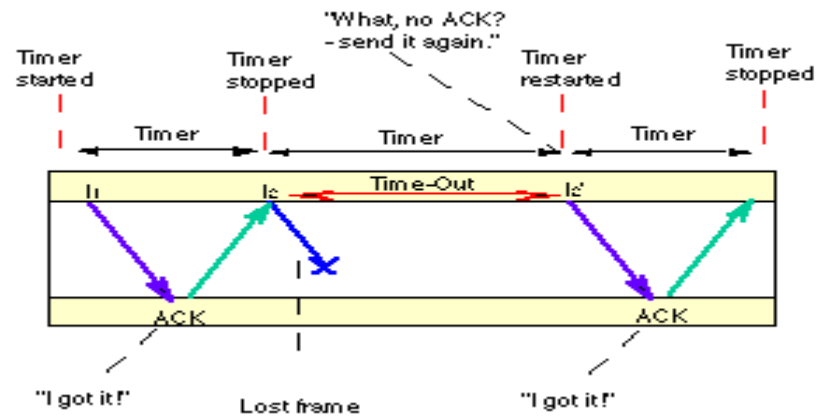
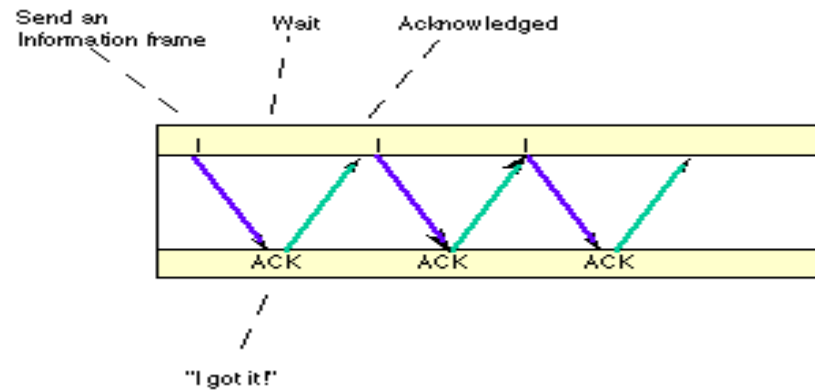
Протоколът с 1-битов пореден номер се нарича още **Stop-And-Wait** (“спри и чакай”).

- Предавателят изпраща **един кадър**;
- чака **потвърждение** за един **RTT (round trip time)** - времето, необходимо на сигнала за отиване до приемника и връщане обратно;
- след получаване на потвърждение изпраща **следващ кадър**.

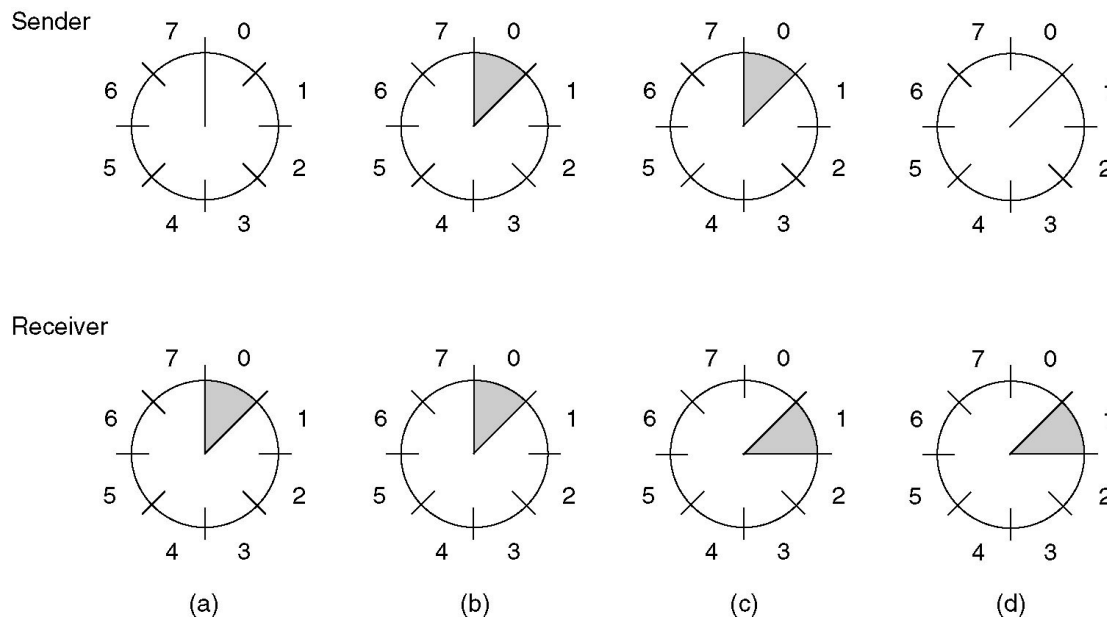
Не е ефективен. Само един кадър се предава в даден момент.

Чакането е толкова по-голямо, колкото по-бавна линията — напр. сателитна връзка.

Stop and Wait (примери: ОК, загуба)



Sliding Window Protocols



Прозорец с размер 1 и с 3-битов пореден номер.

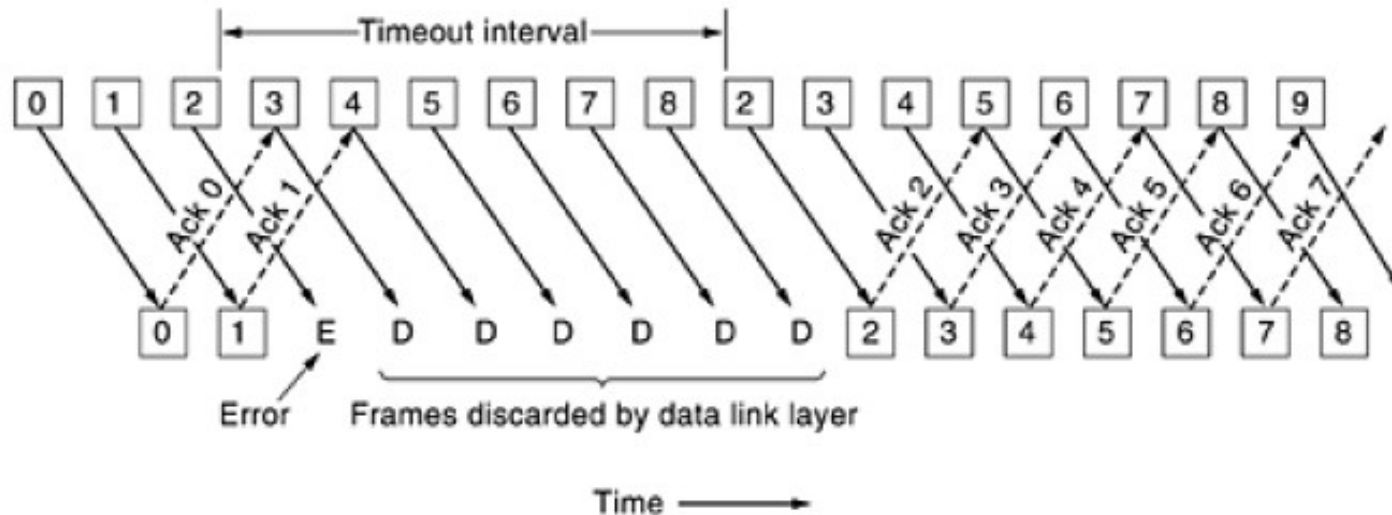
(a) Отначало.

(b) След изпращане на първи кадър.

(c) След получаване на първи кадър.

(d) След получаване на потвърждение.

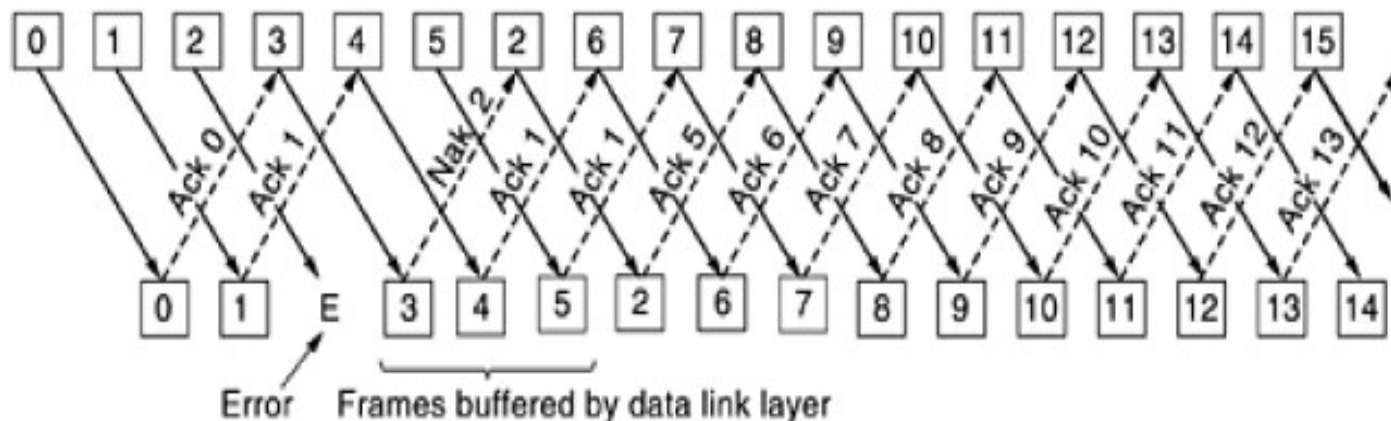
При грешка в прозореца. Go-Back-n.



Ако липсва и един кадър, напр. 2, приемникът изхвърля всички следващи до изчерпване на прозореца.

Предавателят трябва да ги предаде повторно. Губи се пропускателна способност.

При грешка в прозореца. Selective Repeat.



Повторно се изпращат само изгубените и/или повредени кадри.

Приемникът буферира всички кадри след изгубения/повредения.

Когато предавателят забележи проблема, (няма АСК за определен time-out), кадърът се предава наново.

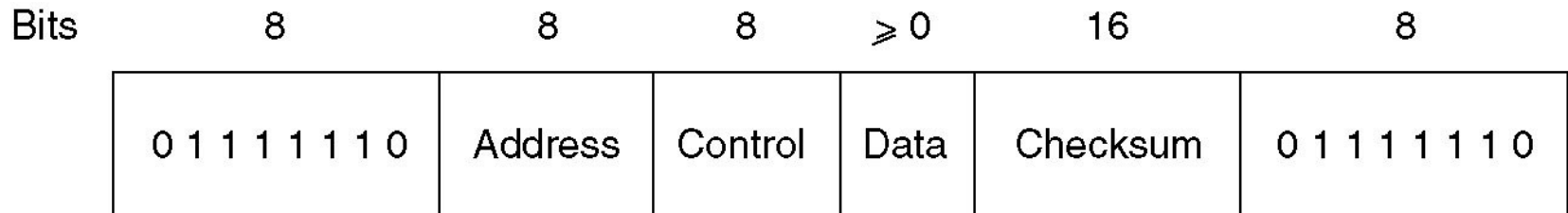
High-Level Data Link Control

Първият протокол на канално ниво, който се използва в IBM е **SDLC** (synchronous data link control).

По-късно организацията по стандартизация ISO разработва на базата на SDLC протокола **HDLC** (high-level data link control).

И двата протокола са **битово-ориентирани** и използват **вмъкване на битове** за правилно идентифициране на кадрите.

Форматът на кадъра в HDLC е следния:



HDLC

В началото и в края на кадъра са **флаговете** за маркиране на границите на кадъра.

Полето **Address** се използва при многоточкови канали (multipoint) и чрез него се идентифицира получателя на кадъра.

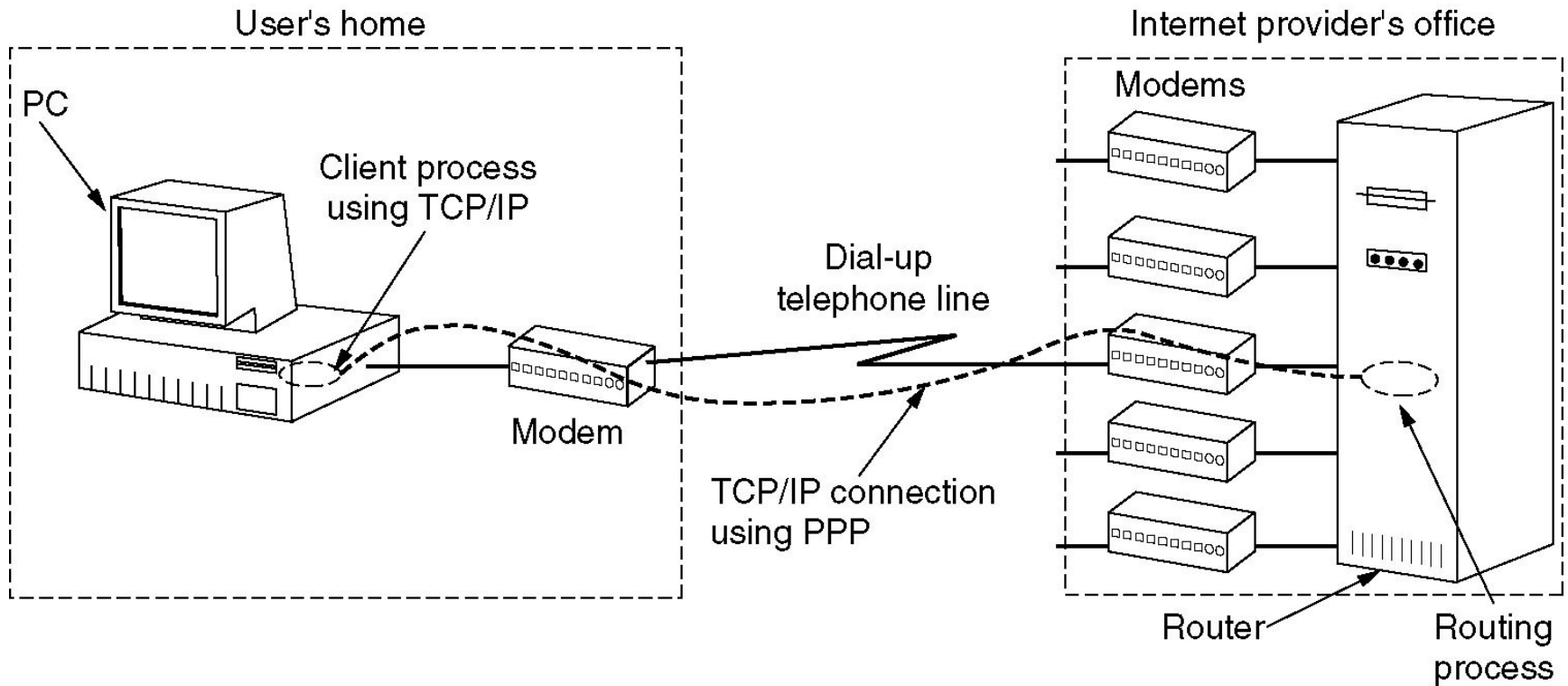
Полето **Control** се използва за номериране на кадрите, за потвърждения и др.

Полето **Data** съдържа данните на кадъра. По принцип има неограничена дължина.

Полето **Checksum** е контролната сума на кадъра (използват се циклични кодове).

Минималната дължина на кадъра, без да се включват флаговете за начало и край е **32 бита**.

PPP (Point-to-Point Protocol)



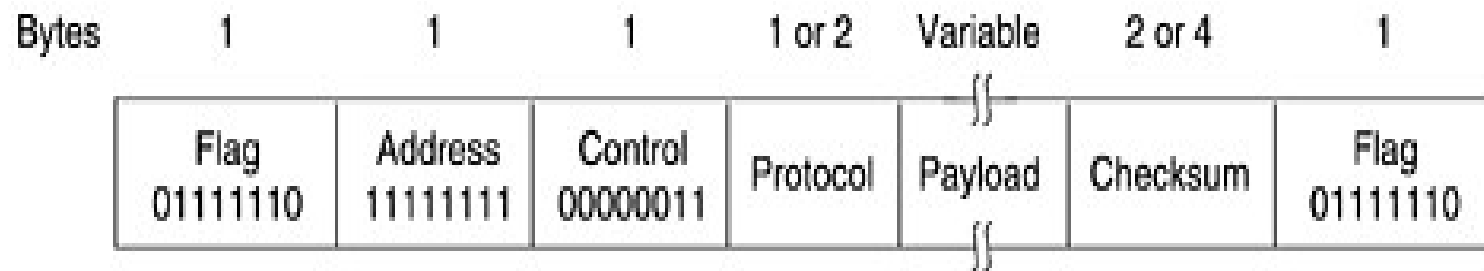
Свързване към internet по PPP.

Протокол PPP

Протоколът PPP е протокол за двуточкова връзка. Този протокол се използва за свързване на домашни компютри до доставчици на Интернет услуги по телефонна линия.

Протоколът PPP е байтово-ориентиран и за идентифициране на кадрите се използва техниката вмъкване на байтове.

Форматът на кадъра е наследен от HDLC:



Протокол PPP

При PPP няма индивидуални адреси на станциите, затова полето *Address* съдържа 1111111, което означава адресите на всички станции.

Полето *Control* съдържа 00000011, което означава *unnumbered*-кадър. С други думи, PPP не осигурява надеждно предаване чрез номера на кадрите и потвърждения.

Полето *Protocol* съдържа идентификатор на протокол, който указва как да се интерпретира полето *Payload*, в което се помещава съответния пакет.

Максималната дължина на *Payload* е 1500 байта.

Протокол RPP

Дължините на полетата *Protocol* и *Checksum* се договарят при установяването на съединение. След установяване на съединение, двете страни се договарят за мрежовите протоколи, които ще се използват. След това започват да се предават кадрите с данни, като полето *Protocol* съдържа идентификатор на един от уговорените мрежови протоколи, а *Payload* съдържа съответната дейтаграма.

PPP фази

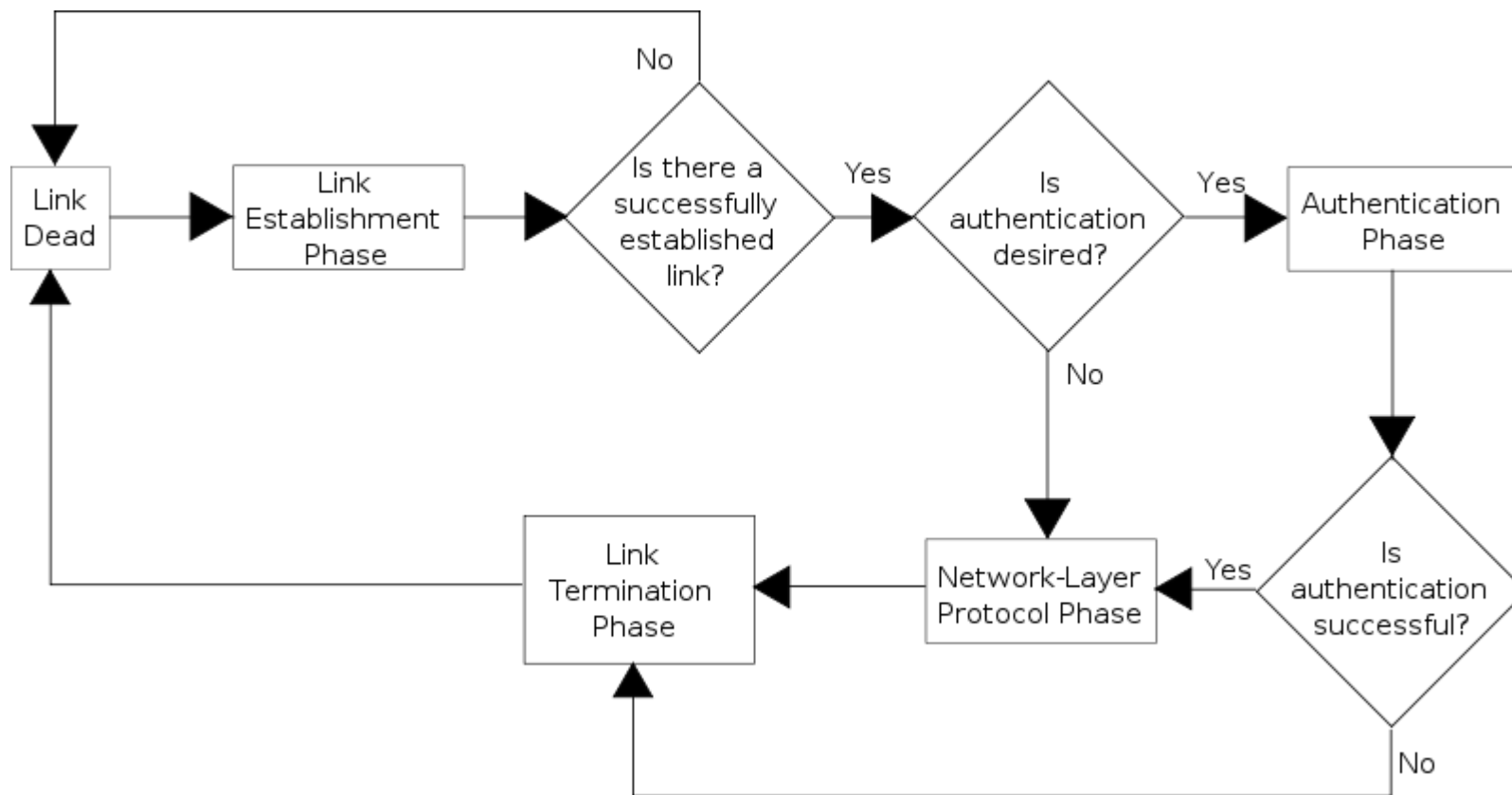


Схема на установяване на връзката.

LCP

Line Control Protocol (LCP):

автоматично конфигуриране на срещуположните интерфейси:

дължина на кадъра, ESC символи;

Проверка на линията за грешки с произволни числа (**magic numbers**). Ако линията е дадена накъсо, възелът получава LCP съобщение със своя си magic number, вместо да получи magic number на съседа;

Компресия.

Последвани евентуално от **аутентикация**.

Аутентикация

Съседите си обменят съобщения за аутентикация.

Имаме два варианта:

Password Authentication Protocol (**PAP**) и
Challenge Handshake Authentication Protocol
(**CHAP**).

PAP

PAP предава пароли в явен ASCII текст по мрежата, затова е несигурен.

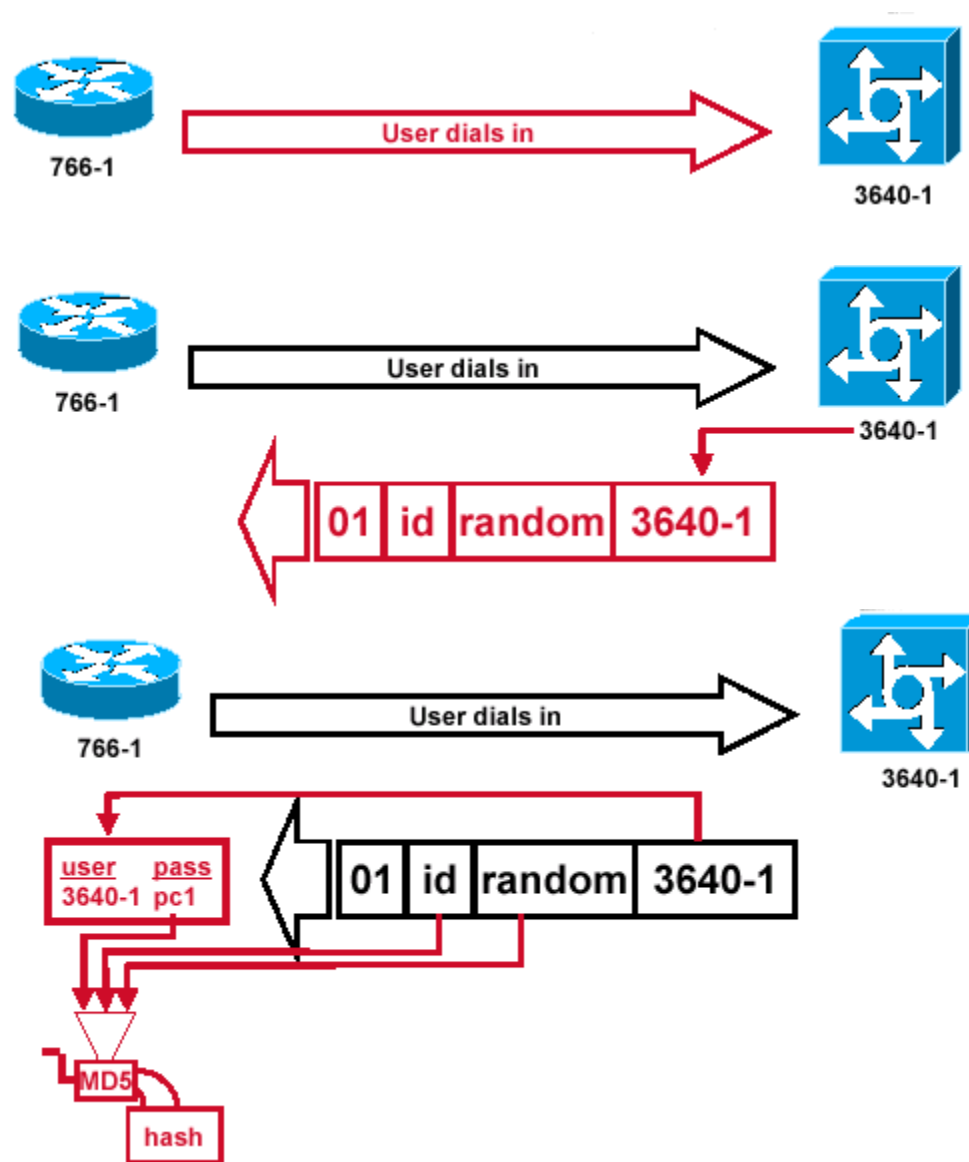
- * Клиент изпраща `username` и `password`
- * Сървърът връща:
 `authentication-ack` (ако е ОК) или
 `authentication-nak` (в противен случай).

SHAP

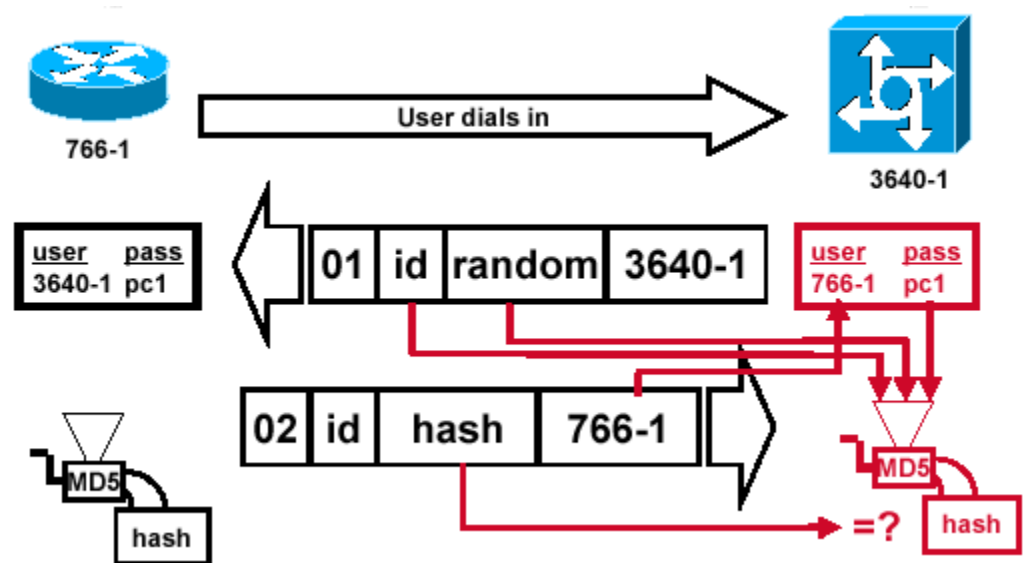
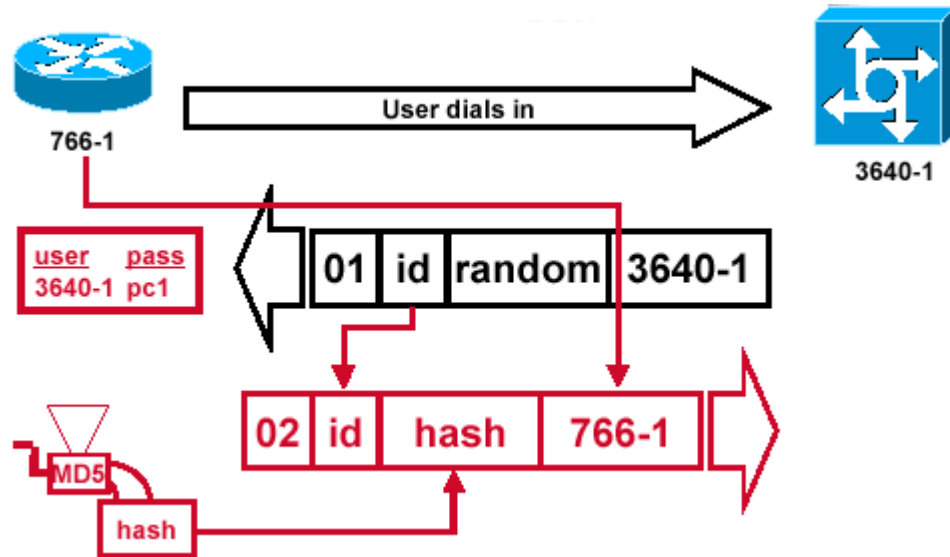
SHAP периодически проверява идентичността на клиента чрез **three-way handshake**. При установяване на сесията и през произволни интервали от време след това. Проверката се базира на **споделена “тайна”** (напр. Паролата на потребителя).

1. **Сървърът** изпраща "**challenge**" съобщение към клиента.
2. **Клиентът** отговаря с число, изчислено с помощта на еднопосочна хеш функция, напр. **MD5 checksum hash**.
3. **Сървърът** сравнява този хеш със своя. Ако съвпадат, следва **acknowledge**; в противен случай връзката се прекъсва.
4. През **произволни интервали** сървърът изпраща ново предизвикателство: стъпки 1-3 се повтарят.

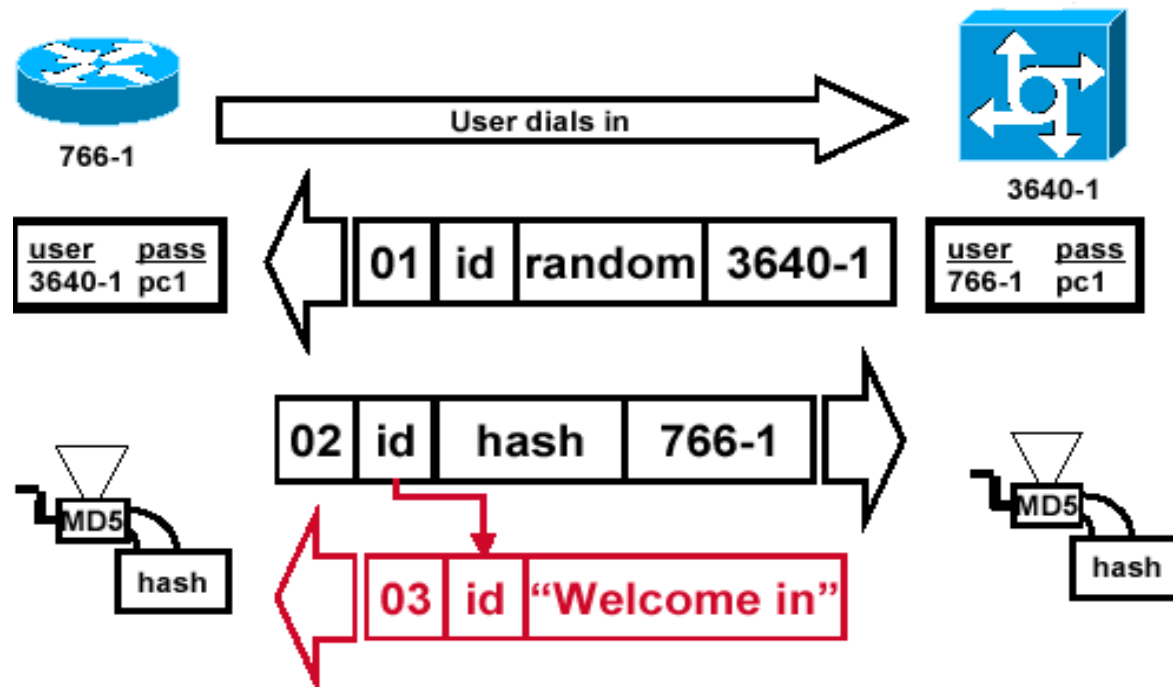
СНАР. Етапи.



СНАР. Етапи.



СНАР. Етапи.



Network Control Protocol

Network Control Protocol (NCP) се стартира след LCP.

Уговаря опции за протокола от **мрежовия слой**, над PPP. NCP са:

Internet Protocol Control Protocol (**IPCP**) за IP,
Internetwork Packet Exchange Control Protocol
(**IPXCP**) за IPX и

AppleTalk Control Protocol за AppleTalk и

IPv6 Control Protocol (IPV6CP) за предаване на IPv6 пакети по PPP линии.

Развитие на PPP – PPPoE

PPPoE, Point-to-Point Protocol over Ethernet опакова PPP кадри вътре в Ethernet кадри.

Използва се при свързвания към Интернет чрез LANs, WLANs или Metro Ethernet мрежи.

Разработена е от UUNET, Redback Networks и RouterWare, стандартизирана е в RFC (Request for Comment) 2516.

Чрез PPPoE потребителите виртуално “набират номера” на отдалечен сървър на провайдера през Ethernet и установяват “point to point” връзка.

PPPoE - стадии

PPPoE се установява на два точно определени стадия:

PPPoE discovery

Традиционните PPP връзки се установяват между две крайни точки, които са предварително изградени.

Но Ethernet мрежите са multi-access, така че преди обмен на PPP контролни пакети за установяване на връзката върху Ethernet, двете страни ще трябва да си научат MAC адресите, за да бъдат закодирани в контролните пакети.

Също така се установява **Session Id**, която се използва при обмена на пакети.

PPP session

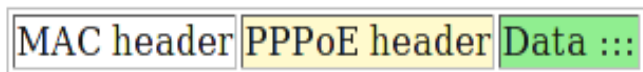
След като са известни MAC адресите и е установена сесията, двете страни имат всичката информация за изграждане на “point-to-point” връзка по Ethernet и обмен на пакети.

PPPoE Frame

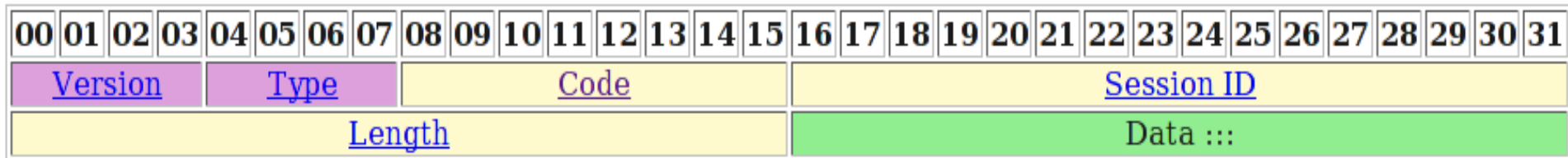
Вмъкването на PPP “заглавие” води до намаляване на полето за данни от **1500 на 1492 байта**.

Намалява ефективната скорост:

$$1492:1500 = \mathbf{0.995}$$



PPPoE header:



MPLS

Multiprotocol Label Switching (MPLS) е механизъм за реализация на високопроизводителни телекомуникационни мрежи.

Разработен е от **IETF** (Internet Engineering Task Force).

MPLS работи на OSI "**Слой 2.5**".

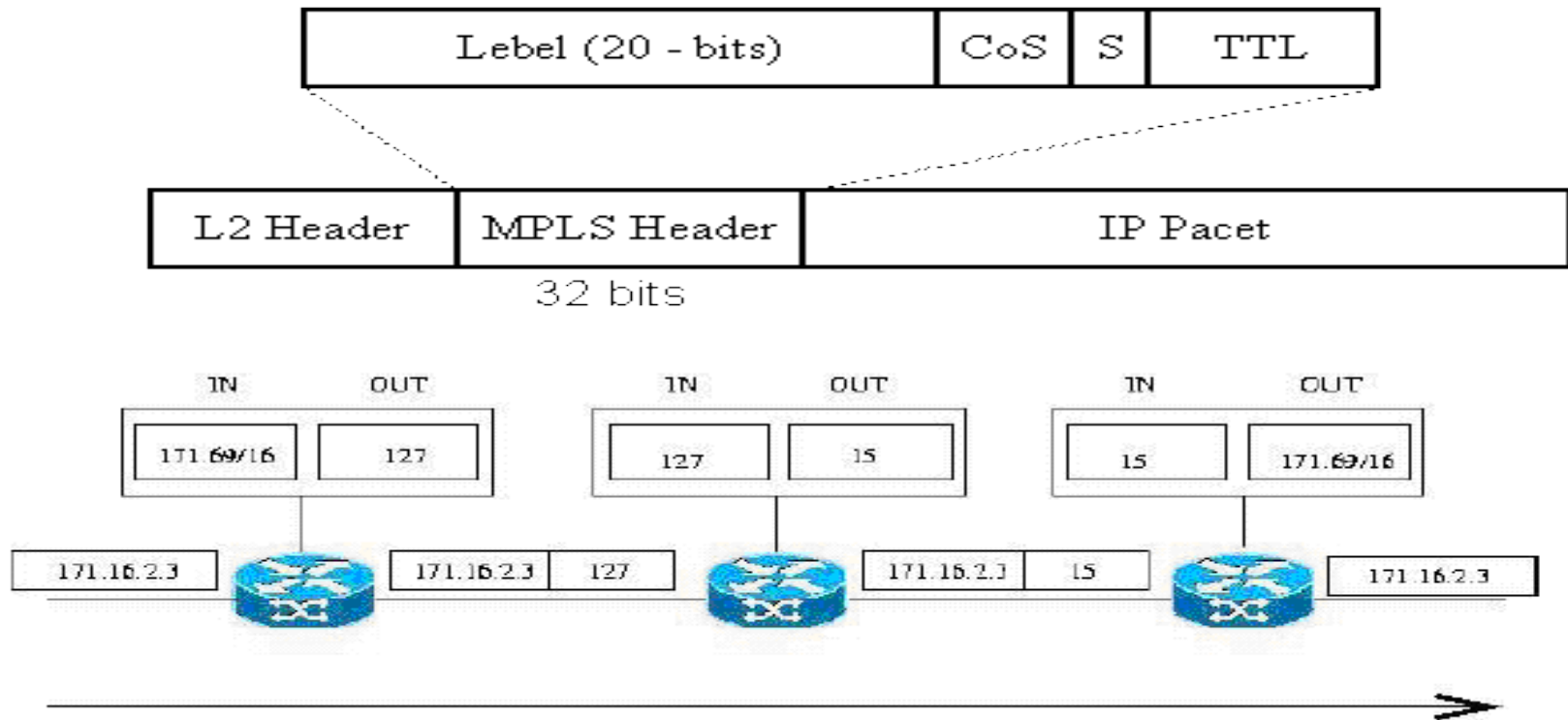
Multiprotocol:

Пренася всякакви видове трафик: **IPv4** и **IPv6**, **Ethernet** и др.

Label Switching:

Данните се направляват от възел на възел с помощта на **етикети**.

MPLS. Layer 2.5.



Фигура 3: Препращане на пакети чрез етикети в MPLS

Етикетът съответства на Forwarding Equivalence Class – **FEC**; Пакети от един и същ FEC се насочват по един и същ начин в MPLS мрежата.

Етикетът се проверява за съвпадение в информационната база с етикети (LIB), за да се определи следващия участък по пътя на пакета.

MPLS и Ethernet

Митовете за MPLS <http://delian.blogspot.com/2006/11/mpls.html>

- QoS в IP – в IP имаме повече приоритети (64 с DSCP, срещу 8 ако използваме Ехр за CoS);
- Layer3 VPN - L3 VPN се прави и с GRE тунели, и с IPSec, и с чист Ethernet. и т.н. и т.н.

MPLS не може да се мери с “чиста” Ethernet мрежа.

Ethernet е по-евтин, същата функционалност, по-висок капацитет и по-висока съвместимост.

Ethernet е достатъчна само заради бързодействието.